

Part 3 进程控制

3.3 Unix进程控制实现

● 创建fork

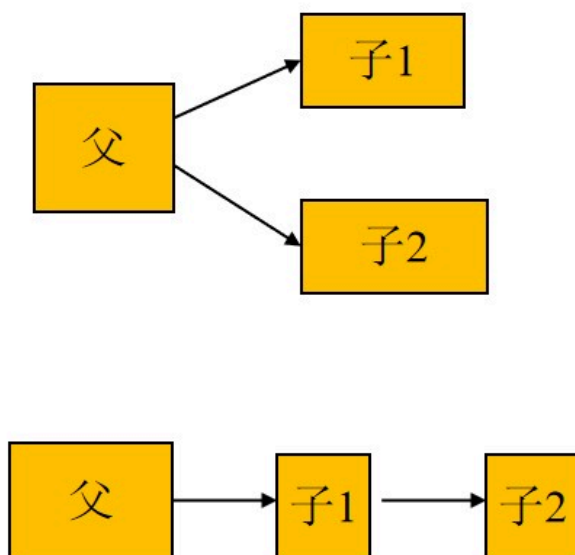
fork(): 创建子进程，系统执行操作：

- ❖ 为子进程分配进程表。
- ❖ 为子进程分配进程标识符。
- ❖ 复制父进程的进程映像，但不复制共享内存区。
- ❖ 增加父进程所打开文件的计数，表示新进程也在使用这些文件。
- ❖ 把子进程置为“就绪”状态。
- ❖ **返回子进程标识符给父进程，把0值返回给子进程。**

Part 3 进程控制

3.3 Unix进程控制实现

●fork应用2



```
main()
{
    if(fork()==0)
        {子1的代码段}
    else
        {if(fork()==0)
        {子2的代码段}
        else
        {父代码段}
        }
}
```

```
main()
{
    if(fork()==0)
        {子1的代码段;
        if(fork()==0)
            {子2的代码段}
        else
            {子1的代码段}
        }
    else
        {父代码段}
}
```

Part 3 进程控制

3.3 Unix进程控制实现

●fork应用3

分析程序的执行结果

```
main()
{  int pid;
   printf("Before fork\n");
   while((pid=fork())!=-1);
   if(pid){
       printf("It is parent process:PID=%d\n", getpid());
       printf("Produce child 'sPID=%d\n" ,pid);}
   else
       printf("It is child process:PID=%d\n",getpid());
   printf("It is parent or child process:PID==%d\n",getpid());
}
```

Part 3 进程控制

3.3 Unix进程控制实现

● 思考题

**不算 main 这个进程自身，到底创建了多少个进程？
(课后思考)**

注意：逻辑运算的短路法则

```
int main(int argc, char* argv[])
{
    fork();
    fork() && fork() || fork();
    fork();
}
```

调用fork函数创建进程，父子进程会获取不同的返回值。下列关于fork函数的说法正确的是（ ）。

- ☒ A 调用成功，子进程中返回子进程的pid，父进程中返回0
- ☐ B 调用成功，子进程和父进程都返回0
- ☒ C 创建失败，父进程中返回-1
- ☐ D 创建成功，父子进程返回的值相同都是子进程的pid

关于fork函数调用成功之后，父子进程的执行，描述正确的是（ ）。

- ☐ A 子进程从调用语句之后继续执行
- ☐ B 子进程从程序第一条语句执行
- ☒ C 父子进程执行的语句完全相同
- ☐ D 父子进程执行顺序为先父进程再子进程

下列程序中创建进程成功时，程序可能的输出结果为（ ）。

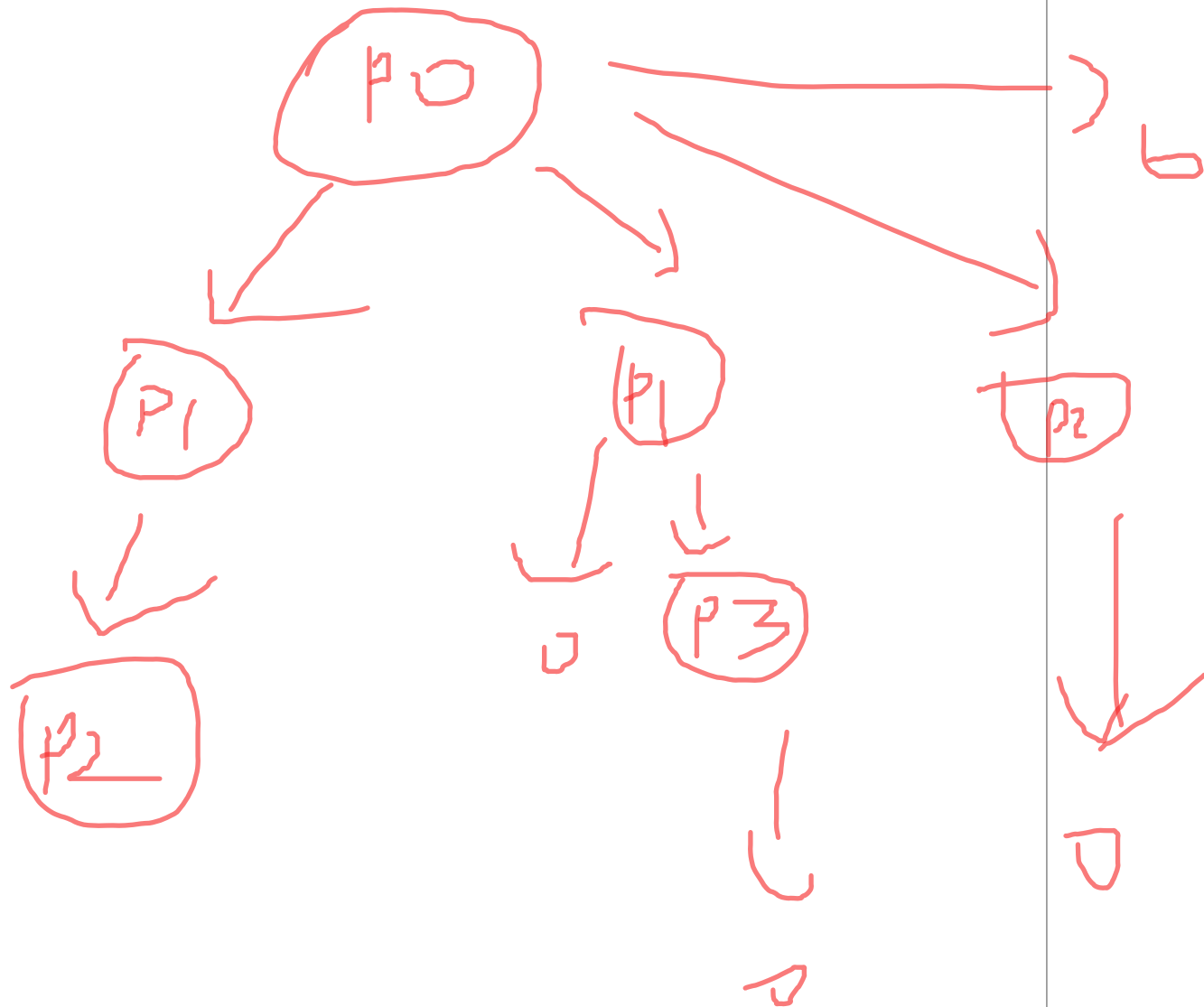
```
main ()
{ printf("hello\n");
  if(p=fork()) printf("world\n");
  esle printf("china\n");
}
```

- ☐ A hello换行 world换行
- ☒ B hello换行 world换行 china换行
- ☐ C hello换行 china换行
- ☐ D hello换行 world换行 hello换行china换行

5分

画出程序生成的进程树

```
main()
{ int p;
  if ((p=fork())==0)
    { fork();exit(0);}
  else
    { fork();
      fork();
    }
  fork();
}
```



Part 3 进程控制

3.3 Unix进程控制实现

- 进程映像的改换—exec

exec: 更换进程执行代码, 更换正文段, 数据段

调用格式: exec (文件名, 参数表, 环境变量表)

exec系列函数有execl、execv、execlp、execvp等

Part 3 进程控制

3.3 Unix进程控制实现

● 进程映像的改换—exec

分析程序的执行结果

```
int main()
{ int pid;
  printf("Now this is process 1.\n");
  if ((pid=fork())==-1) { perror("fork failed"); exit(1); }
  else if(pid==0)
  { printf("This is child process.\n");
    if(execl("/bin/ps","ps","-ef",NULL)<0) perror("execl of ps failed");
    printf("this is subprocess.\n");
    exit(0); }
  else if(pid>0) printf("this is process1(parent process).\n");
  return 0;
}
```

Part 3 进程控制

3.3 Unix进程控制实现

- 进程终止—exit

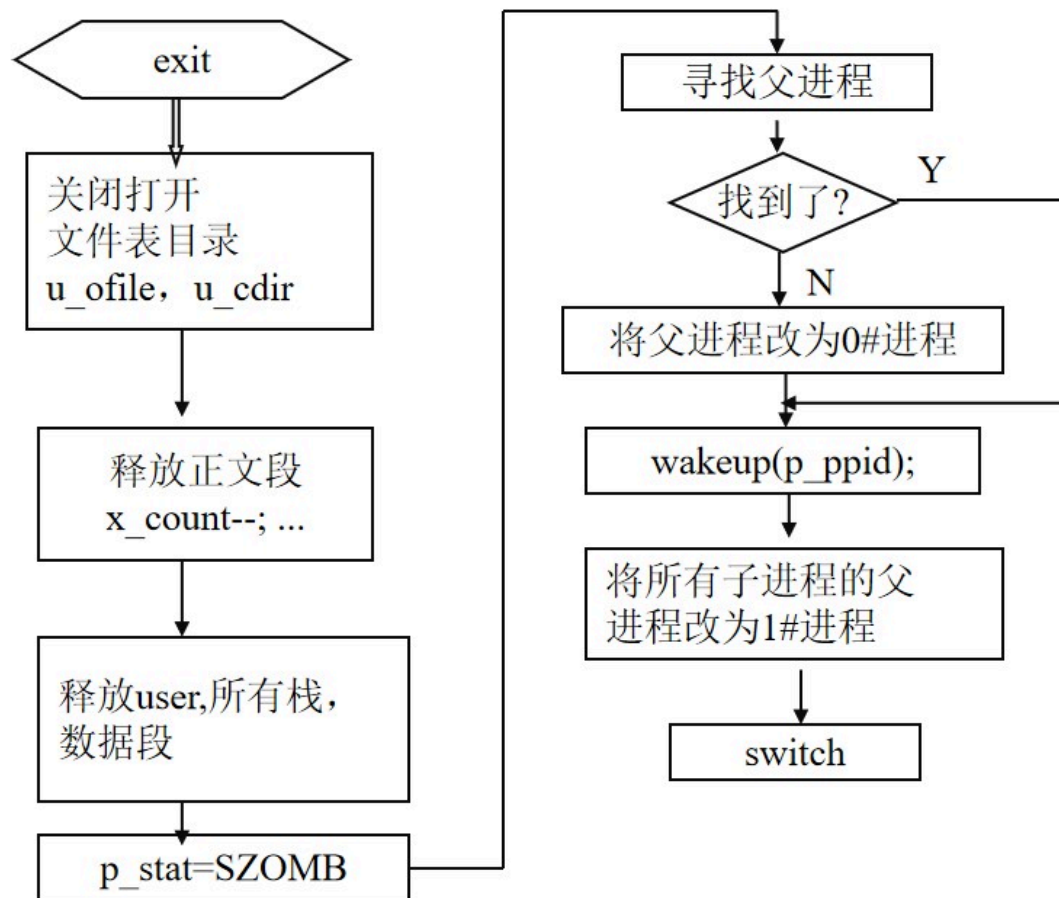
exit: 进程的自我终止

当一个进程执行结束后，进程可以调用exit自我终止。终止时它放弃占用的所有资源，**清除缓冲区**，处于等待父进程善后处理状态。

Part 3 进程控制

3.3 Unix进程控制实现

- 进程终止—exit



Part 3 进程控制

3.3 Unix进程控制实现

- 等待子进程终止—wait

wait: 父进程等子进程的终止

对处于等待善后处理状态的子进程进行善后处理。在用户态进程中，父进程调用wait(status)等待它的一个子进程终止。

它的子进程还没有终止，则它进入阻塞态。

Part 3 进程控制

3.3 Unix进程控制实现

- wait应用

```
int main()
{
    int proc_id;
    while((proc_id=fork())!=-1);
    if (proc_id)
    { proc_id=wait(0);
      printf("the child process has finished.\n");
      printf("The child process PID=%d\n",proc_id);
    }
    else{
        printf("In child process.\n");
        exit(0);
    }
    return 0;
}
```

```
#include<sys/wait.h>
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
```

Part 4 进程通信



学习目标

- 能够识别进程通信的几种类型
- 能够理解Unix/Linux进程通信的实现方式
- 能够应用Unix/Linux进程通信方式分析设计程序

Part 4 进程通信

4.1 进程通信概念

-  进程通信是指进程之间的信息交换
-  **低级进程通信**：进程的同步和互斥
 - 效率低
 - 通信对用户不透明
-  **高级进程通信**
 - 使用方便
 - 高效地传送大量数据

Part 4 进程通信

4.2 进程通信类型

-  信号
-  管道
-  消息队列
-  共享存储器

Part 4 进程通信

4.3 Unix/Linux进程通信类型



Part 4 进程通信

4.3 Unix/Linux进程通信类型

- 信号

- 信号概念

- 软件中断

- 处理异步事件的机制

- 信号名（以SIG开头）

Part 4 进程通信

4.3 Unix/Linux进程通信类型

- 信号

信号产生

按终端键, 硬件异常, kill(2)函数, kill(1)命令, 软件条件, ...

Part 4 进程通信

4.3 Unix/Linux进程通信类型

● 信号

名称	说明	默认操作
SIGABRT	进程异常终止（调用 abort 函数产生此信号）	
SIGALRM	超时（ alarm ）	
SIGFPE	算术运算异常（除以0，浮点溢出等）	
SIGHUP	连接断开	终止
SIGILL	非法硬件指令	
SIGINT	终端终端符(Ctrl-C)	终止
SIGKILL	立即结束程序运行（不能被捕捉或忽略）	终止
SIGPIPE	向没有读进程的管道写数据	
SIGQUIT	终端退出符(Ctrl-\)	终止
SIGTERM	终止（由 kill 命令发出的系统默认终止信号）	退出
SIGUSR1	用户定义信号	退出
SIGUSR2	用户定义信号	退出

Part 4 进程通信

4.3 Unix/Linux进程通信类型

● 信号

名称	说明	默认操作
SIGSEGV	无效存储访问（段违例）	
SIGCHLD	子进程停止或退出	忽略
SIGCONT	使暂停进程继续 略	继续/忽略
SIGSTOP	暂停一个进程（不能被捕捉或忽略）	终止
SIGTSTP	终端挂起符(Ctrl-Z) 程	停止进程
SIGTTIN	后台进程请求从控制终端读	
SIGTTOU	后台进程请求向控制终端写	

Part 4 进程通信

4.3 Unix/Linux进程通信类型

- 信号

- 信号处理

- 忽略信号

- 不能忽略的信号: SIGKILL, SIGSTOP、一些硬件异常信号

- 执行系统默认动作

- 捕捉信号

Part 4 进程通信

4.3 Unix/Linux进程通信类型

● 信号

信号发送和接收

Kill: 给一个进程发送信号

pid参数:

pid > 0: 发送信号给进程id为pid的进程。

pid = 0: 发送信号给与自己同组的所有进程。

pid = -1: 发送系统内所有进程

pid < -1: 发送给进程组号为-pid的所有进程。

```
#include <sys/types.h>
```

```
#include <signal.h>
```

```
int kill(pid_t pid, int sig);
```

(Returned Value: 0 if success, -1 if

failure)

Part 4 进程通信

4.3 Unix/Linux进程通信类型

- 信号

- 信号发送和接收

signal: 设置收到信号时信号的处理方式

“handler” 参数:

用户指定的函数名

SIG_DFL

SIG_IGN

```
#include <signal.h>
typedef void
(*sighandler_t)(int);
sighandler_t signal(int
signum, sighandler_t
handler);
```

Part 4 进程通信

4.3 Unix/Linux进程通信类型

- 信号

信号使用例1——默认方式

```
int main()
{ void fun( );
  int i;
  signal(SIGINT,SIG_DFL);
  for(i=0;i<10;i++)
  {
    printf("second:%d\n",i);
    sleep(1);
  }
  return 0;
}

void fun()
{ system("date"); }
```

Part 4 进程通信

4.3 Unix/Linux进程通信类型

- 信号

信号使用例1——忽略方式

```
int main()
{
    void fun();
    int i;
    signal(SIGINT,SIG_DFL);
    for(i=0;i<10;i++) {
        printf("second:%d\n",i);
        sleep(1);
    }
    return 0;
}

void fun()
{ system("date"); }
```

Part 4 进程通信

4.3 Unix/Linux进程通信类型

- 信号

- 信号使用例1——捕捉方式

```
int main()
{ void fun();
  int i;
  signal(SIGINT,fun);
  for(i=0;i<10;i++)
  {
    printf("second:%d\n",i);
    sleep(1);
  }
  return 0;
}

void fun()
{ system("date"); }
```


Part 4 进程通信

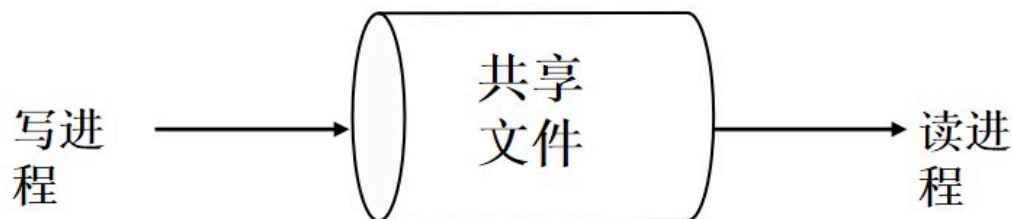
4.3 Unix/Linux进程通信类型

- 无名管道

管道原理

❖管道(pipe)是连接读写进程的一个特殊文件, 允许进程按先进先出方式传送数据,也能使进程同步执行操作。

❖发送进程以字符流形式把大量数据送入管道, 接收进程从管道中接收数据, 所以叫管道通信。



Part 4 进程通信

4.3 Unix/Linux进程通信类型

- 无名管道

UNIX管道相关系统调用

通过pipe系统调用创建无名管道，得到两个文件描述符，分别用于写和读。

- **int pipe(int fildes[2]);**

文件描述符fildes[0]为读端，fildes[1]为写端；

- **int read(int fd, char * buf, unsigned nbyte)**

从fd所指示的文件中读出nbyte个字节的数据，送至由指针buf所指示的缓冲区中。

- **int write(int fd, char * buf, unsigned nbyte)**

把nbyte 个字节的数据，从buf所指向的缓冲区写到由fd所指向的文件中。

Part 4 进程通信

4.3 Unix/Linux进程通信类型

● 无名管道

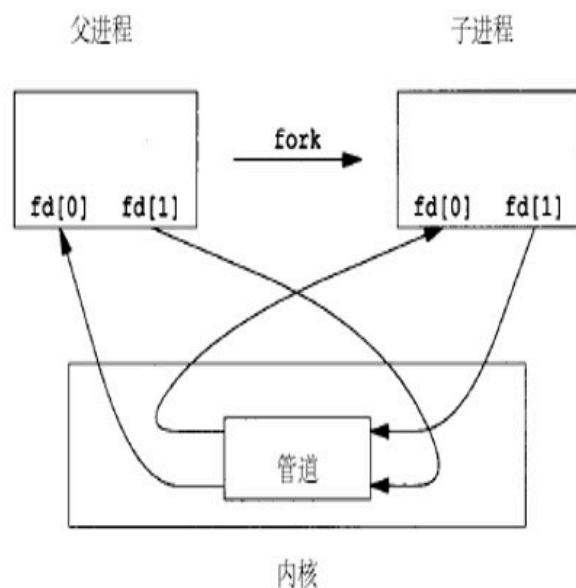


图14-2 fork之后的半双工管道

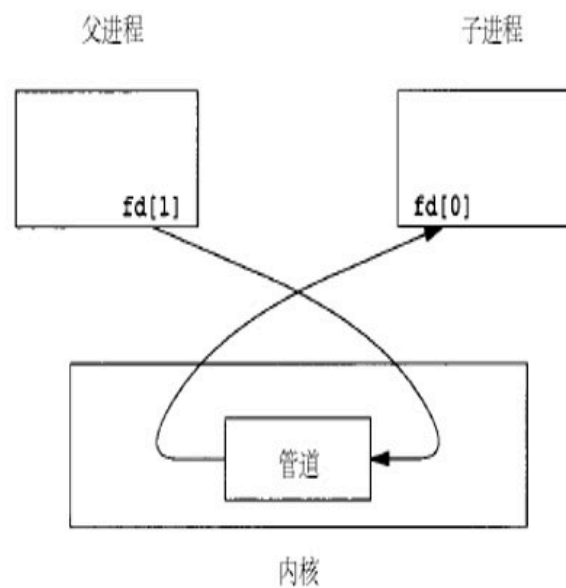


图14-3 从父进程到子进程的管道

Part 4 进程通信

4.3 Unix/Linux进程通信类型

● 无名管道 pipe使用例

建立一个pipe,父进程生成一个子进程,子进程向pipe中写入字符串,父进程从pipe 中读出该字符串。

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/wait.h>
```

```
int main()
{ int p,fd[2]; char buf[30],s[30];
  pipe(fd);
  while((p=fork())!=-1);
  if (p==0) { close(fd[0]);
              sprintf(buf,"this is an example");
              write(fd[1],buf,30);
              exit(0);}
  else{ wait(0); close(fd[1]);
        read(fd[0],s,30);
        printf("%s\n",s);}
}
```

Part 4 进程通信

4.3 Unix/Linux进程通信类型

- 消息队列

原理

